K3s Setup and Manage Cluster

Absolutely, I understand what you're aiming for: a **modern, production-like K3s cluster** on **two VMs** (1 master, 1 agent) with:

- 🧠 Proper Kubernetes management setup
- 📊 Full monitoring (Grafana + Prometheus + metrics)
- ✅ Best practices as used by experts
- 🎯 Jenkins already running (we'll exclude it from redeployment)

🛠️ Step-by-Step K3s Cluster Setup with Full Monitoring

✅ [STEP 1] — Prerequisites

✅ On Both VMs (Master & Agent):

```
sudo apt update && sudo apt install -y curl
sudo hostnamectl set-hostname <master-or-agent-hostname>
```

📌 Ensure:

- Both VMs can ping each other by IP
- You open required ports:
  - TCP 6443 (K8s API)
  - TCP 10250 (Kubelet)
  - TCP 9100 (Node Exporter)
  - TCP 3000 (Grafana)
  - TCP 9090 (Prometheus)

✅ [STEP 2] — Install K3s

🔷 On Master Node (VM1):

```
curl -sfL https://get.k3s.io | sh -
```

Get the token:

```
sudo cat /var/lib/rancher/k3s/server/node-toke
```

Verify:

```
kubectl get nodes
```

🔷 On Agent Node (VM2):

Replace `<MASTER_IP>` and `<NODE_TOKEN>`:

```
curl -sfL https://get.k3s.io | K3S_URL=https://<MASTER_IP>:6443 K3S_TOKEN=<NODE_TOKEN> sh -
```

✅ [STEP 3] — Enable kubectl Access on Master

```
mkdir -p ~/.kube
sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config
sudo chown $USER:$USER ~/.kube/config
```

✅ [STEP 4] — Install Helm (Master Node)

```
curl https://raw.githubusercontent.com/helm/helm/master/scripts/get-helm-3 | bash
```

✅ [STEP 5] — Deploy Monitoring Stack (Prometheus + Grafana)

🔷 Add Prometheus Helm Chart Repo

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

🔷 Install Monitoring Stack in `monitoring` namespace

```
kubectl create namespace monitorin
helm install kube-monitor prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

🛠️ This installs:

- **Prometheus**
- **Alertmanager**
- **Grafana**
- **Node Exporter**
- **Kube-State-Metrics**

✅ [STEP 6] — Access Grafana

Port-forward temporarily (for testing)

```
kubectl port-forward svc/kube-monitor-grafana -n monitoring 3000:80
```

Go to:

http://localhost:3000

Default login:

- **User**: `admin`
- **Password**: `prom-operator`

👉 Change password after first login.

✅ [STEP 7] — Set Up Dashboards in Grafana

Grafana comes with many Kubernetes dashboards by default:

- Node Exporter full metrics (CPU, memory, disk)
- Pod resource usage
- Namespace usage
- Deployment health
- Kubelet metrics
- Scheduler metrics

> 📍 Tip: Go to Dashboards > Manage → you'll find auto-imported dashboards already available.

✅ [STEP 8] — Optional: Expose Grafana via NodePort

```
kubectl patch svc kube-monitor-grafana \
  -n monitoring \
  -p '{"spec": {"type": "NodePort"}}'
```

Get the port:

```
kubectl get svc kube-monitor-grafana -n monitoring
```

Then access:

```
http://<AGENT_NODE_IP>:<NodePort>
```

Verify the update : if can’t access to node port 3000

Use `kubectl patch` command (no editing in text editor)

Run this command **directly** on your master node terminal:

```
kubectl -n monitoring patch svc kube-monitor-grafana -p '{"spec": {"type":
"NodePort","ports":[{"port":80,"targetPort":3000,"node
```

```
kubectl -n monitoring get svc kube-monitor-grafana
```

You should see:

```
TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
NodePort   10.43.xxx.xxx  <none>        80:32000/TCP   XXm
```

✅ [STEP 9] — Verify Metrics Are Collecte

Check:

```
kubectl get pods -n monitoring
```

Look for:

- `prometheus-kube-monitor-*`
- `grafana-*`
- `kube-state-metrics-*`
- `node-exporter-*` on **each node**

Also try:

```
kubectl top node
kubectl top pod -A
```

✅ [Optional] Enable Logging (Loki + Promtail)

Would you like to also:

- Collect logs from Jenkins and pods?
- View logs in Grafana?

If yes, I can walk you through installing **Loki + Promtail** next.

✅ Summary: What You've Set Up

Component	Role
K3s Master	Controls the cluster (scheduler, kube-apiserver)
K3s Agent	Runs Jenkins and app pods
Prometheus	Scrapes K8s, node, and pod metrics
Grafana	Displays dashboards for real-time visibility
Node Exporter	Node-level metrics (CPU, RAM, disk)
Kube-State-Metrics	Kubernetes object health/status

🔧 Step-by-step: Install Portainer CE on K3s (NodePort mode)

Run the following on your **master node**:

```
# Step 1: Create namespace
kubectl create namespace portainer

# Step 2: Add Helm repo
helm repo add portainer https://portainer.github.io/k8s/

# Step 3: Update Helm repos
helm repo update

# Step 4: Install Portainer with NodePort so you can access it
helm install portainer portainer/portainer \
  --namespace portainer \
  --set service.type=NodePort \
  --set service.nodePort=30777 \
  --set service.port=9000 \
  --set persistence.enabled=tru
```

✅ This will install Portainer and expose the web UI on:

```
http://<your-master-node-ip>:30777
```

🔍 Check it's working:

```
kubectl get svc -n portainer
```

Look for output like:

```
NAME       TYPE       CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
portainer  NodePort   10.43.x.x     <none>        9000:30777/TCP  1m
```

👨‍💻 Now go to browser:

```
http://<your-master-node-IP>:30777
```

- Set your **admin password**
- Choose **Kubernetes environment**
- You're ready to manage everything via Portainer 🎉

Here's how to install Headlamp manually via Helm from source:

1. Clone the Headlamp repo:

```
git clone https://github.com/headlamp-k8s/headlamp.git
cd headlamp/charts/headlamp
```

1. Create the namespace (if you haven't):

```
kubectl create namespace headlamp
```

1. Install via Helm from the local chart directory:

```
helm install headlamp . -n headlamp --set ingress.enabled=false --set servic
e.type=NodePort
```

Alternative: Use the official Headlamp manifests (YAML) directly

You can also apply their manifest file:

```
kubectl create namespace headlamp
kubectl apply -f https://raw.githubusercontent.com/headlamp-k8s/headlamp/
master/deploy/headlamp.yam
```

Next steps to access Headlamp:

1. **Get the NodePort:**

```
export NODE_PORT=$(kubectl get --namespace headlamp -o jsonpath="{.sp
ec.ports[0].nodePort}" services headlamp)
```

1. **Get the node IP address:**

```
export NODE_IP=$(kubectl get nodes --namespace headlamp -o jsonpath="{.
items[0].status.addresses[0].address}")
```

1. **Print the URL to access Headlamp:**

```
echo http://$NODE_IP:$NODE_PORT
```

Open that URL in your browser.

To get your access token:

```
kubectl create token headlamp --namespace headlamp
```

Use this token to log into Headlamp UI.

Summary:

- Helm install from local chart worked perfectly.
- The raw YAML URL you tried is deprecated/removed, so ignore that.
- Just use the URL and token you got from Helm install notes.